points that should belong to different clusters (for example the 4 clusters in the top-right quadrant) are instead grouped together. As a result, points that are far away from each other, are all in the same cluster. And if we think about the very definition of k-means, its target is minimizing the squared Euclidean distance of the points to their centroids. When too few centroids are created (like in the example) this will force many points to be assigned to a single centroid, even if far away… therefore, the average distance of points in each cluster from their centroid will be higher than if we chose the right number of centroids!

If we run the algorithm several times, with different values for the number of centroids, we can compute the average (or median) distance of points from their centroids, and choose the lower value: is that good enough?

Well, we are close, but not quite… if you think about it, whatever is the value of k (the number of centroids) we tested last, if we choose k+1, with one more centroid the average distance will drop a little further, because each centroid will have fewer and closer points in its cluster: if we bring this to an extreme, by choosing k=n (one centroid per point in the dataset) we will obtain an average distance-to-centroid equal to zero (because each point will have its own centroid).

How can we counter-balance this? The truth is we can't really balance it easily, but there is an easy empiric method that helps us choosing the best value: we'll see that in a minute.

Before that, let's make another important point clear: the distance of points from centroids is only one of many